Two-Way Traffic Light Control System using Arduino and RGB LEDs

1. Aim of the Project

The aim of this project is to design and implement a two-way traffic light control system using two RGB LEDs interfaced with an Arduino Uno. The LEDs simulate standard traffic signals for two opposite directions with proper timing for Red, Yellow, and Green lights.

---

2. Purpose of the Project

The purpose of this project is to understand the concept of traffic light control and implement it using Arduino. It focuses on using RGB LEDs to represent Red, Yellow and Green signals and ensures smooth traffic flow simulation. This project also demonstrates embedded system skills like LED interfacing, timing control, and simulation using Tinkercad.

---

3. Application of the Project

- Simulation of real-world traffic light systems.
- Educational tool to understand timing and sequencing in embedded system.
- Foundation for smart traffic management systems using sensors.
- Useful for teaching beginners about digital output and timing control In Arduino.

4. Components Required

S.NO.	Quantity	Components
1.	1	Arduino Uno
2.	2	RGB LED (Common Cathode)
3.	6	220Ω Resistors
4.	Many	Jumper Wires

---

5. Key Concepts

1. Traffic Light Sequencing:

Traffic signals operate in a predefined sequence: Green → Yellow → Red. This sequence ensures smooth traffic movement and avoids collisions. In this project, the Arduino controls the timing of these lights to simulate real-world traffic behavior.

2. RGB LED Working:

An RGB LED is a light-emitting diode that combines three LEDs (Red, Green, and Blue) in a single package. By varying the intensity or switching ON/OFF different LEDs, we can create multiple colors. For example:

- Red ON + Green ON = Yellow
- Green ON = Green
- Red ON = Red

3. Common Cathode RGB LED:

In a Common Cathode (CC) RGB LED, the longest pin is the cathode (negative terminal), which is connected to GND. The other three pins are connected to Arduino pins through resistors. To turn a color ON, we provide a HIGH signal (5V) to its pin. Example:

- Red pin HIGH → Red light glows
- Green pin HIGH → Green light glows

- Red + Green HIGH → Yellow light glows

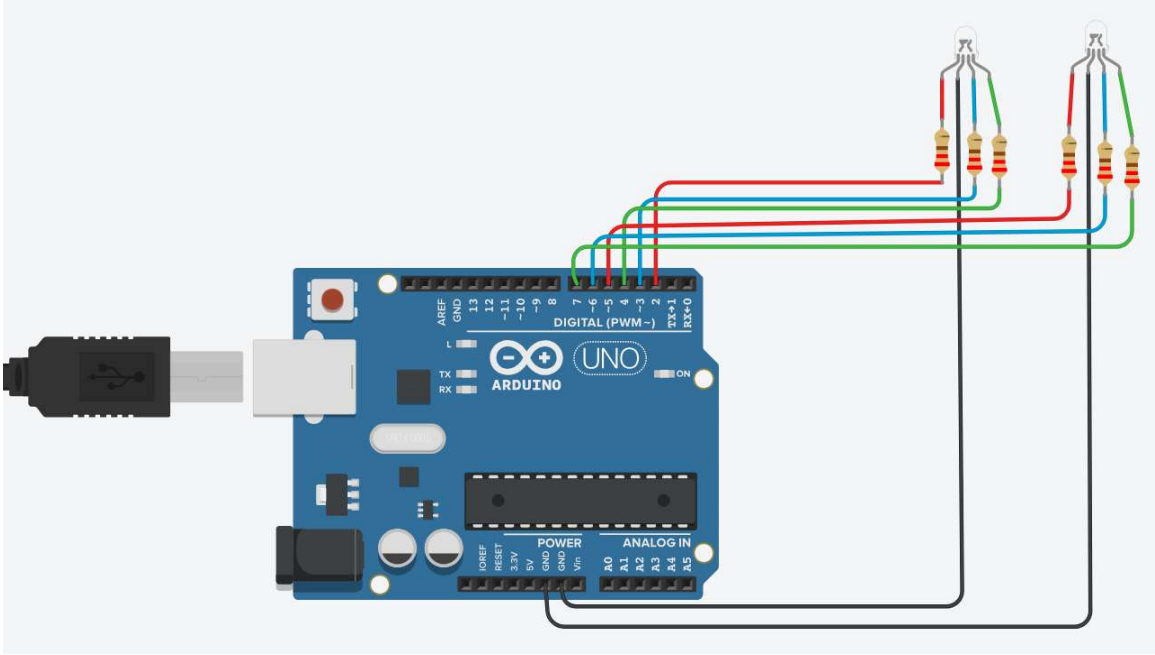
4. Common Anode RGB LED:
In a Common Anode (CA) RGB LED, the longest pin is the anode (positive terminal), which is connected to +5V. The other three pins go to Arduino pins through resistors. To turn a color ON, we provide a LOW signal (0V) to its pin. Example:
- Red pin LOW → Red light glows
- Green pin LOW → Green light glows
- Red + Green LOW → Yellow light glows

5. Difference Between Common Cathode and Common Anode:
- In Common Cathode, HIGH = ON and LOW = OFF.
- In Common Anode, LOW = ON and HIGH = OFF.
Thus, the only difference is how the common pin is wired and how the logic levels work. The internal LEDs are the same.

6. Timing Control in Arduino:
The delay() function is used to control how long each light stays ON. By adjusting the delay values, we can set Green, Yellow, and Red light durations according to real-world requirements. In more advanced systems, millis() can be used for non-blocking timing control.

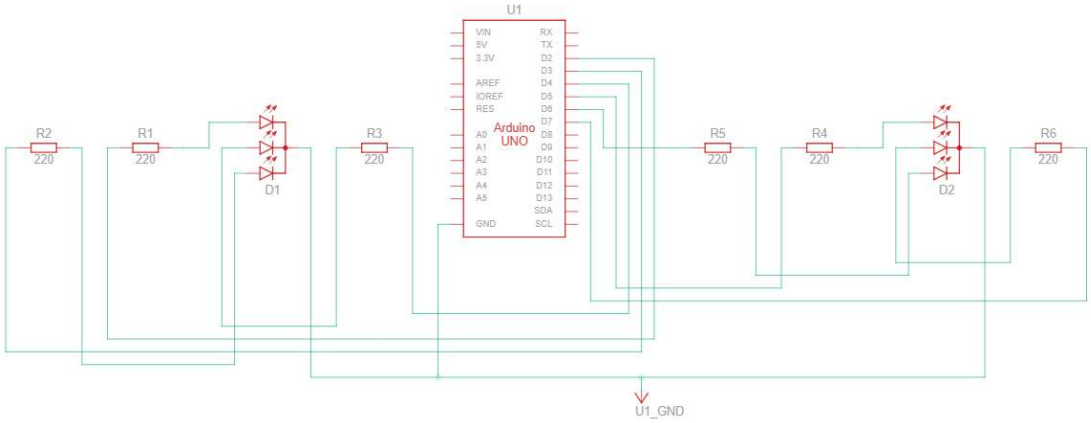
6. Circuit Diagram

The circuit connects two RGB LEDs to Arduino digital pins with resistors. The common pins are connected to GND (for Common Cathode LEDs).



7. Layout (Tinkercad Simulation)

In Tinkercad, connect RGB LEDs with resistors to the respective Arduino pins. The circuit simulates real-time traffic signal operation with alternating Red, Yellow, and Green lights for two directions.



8. Code with Explanation

```
// Two-Way Traffic Light using 2 RGB LEDs (Common Cathode)
// One RGB LED = Direction A
// Another RGB LED = Direction B
// Yellow = Red + Green ON together

// ==== TIME SETTINGS (in milliseconds) ====
const unsigned long T_GREEN  = 5000; // 5 seconds green time
const unsigned long T_YELLOW = 2000; // 2 seconds yellow time

// ==== PIN MAPPING ====
// Direction A (First RGB LED)
const int A_R = 2;  // Red pin of LED A
const int A_G = 4;  // Green pin of LED A
const int A_B = 3;  // Blue pin of LED A (not used here)

// Direction B (Second RGB LED)
const int B_R = 5;  // Red pin of LED B
const int B_G = 7;  // Green pin of LED B
const int B_B = 6;  // Blue pin of LED B (not used here)

// === Helper: Turn ON or OFF any color ===
// For Common Cathode: HIGH = ON, LOW = OFF
void LED_ON (int pin)
{
  digitalWrite(pin, HIGH);
}
void LED_OFF(int pin)
{
  digitalWrite(pin, LOW);
}

// === Function to set color of one RGB LED ===
// r,g,b = 1 (ON) or 0 (OFF)
void setColor(int rPin, int gPin, int bPin, int r, int g, int b)
{
  if (r) LED_ON(rPin); else LED_OFF(rPin);
  if (g) LED_ON(gPin); else LED_OFF(gPin);
  if (b) LED_ON(bPin); else LED_OFF(bPin);
}

// === Shortcuts for each LED ===
void setA(int r, int g, int b)
{
  setColor(A_R, A_G, A_B, r, g, b);
}
```

```
void setB(int r, int g, int b)
{
  setColor(B_R, B_G, B_B, r, g, b);
}

// === Function to turn OFF both LEDs ===
void allOff() {

  setA(0,0,0);
  setB(0,0,0);
}

void setup() {

  // Define all pins as output
  pinMode(A_R, OUTPUT); pinMode(A_G, OUTPUT); pinMode(A_B, OUTPUT);
  pinMode(B_R, OUTPUT); pinMode(B_G, OUTPUT); pinMode(B_B, OUTPUT);

  allOff(); // Start with all LEDs OFF
}

void loop() {

  // STEP 1: Direction A = GREEN, Direction B = RED
  setA(0,1,0);  // A = Green
  setB(1,0,0);  // B = Red
  delay(T_GREEN);

  // STEP 2: Both = YELLOW
  setA(1,1,0);  // A = Yellow
  setB(1,1,0);  // B = Yellow
  delay(T_YELLOW);

  // STEP 3: Direction A = RED, Direction B = GREEN
  setA(1,0,0);  // A = Red
  setB(0,1,0);  // B = Green
  delay(T_GREEN);

  // STEP 4: Both = YELLOW
  setA(1,1,0);  // A = Yellow
  setB(1,1,0);  // B = Yellow
  delay(T_YELLOW);

  // Loop repeats
}
```

9. Conclusion

This project demonstrates the implementation of a two-way traffic light controller using Arduino Uno and RGB LEDs. It helps to understand timing control, LED interfacing, and real-time simulation of traffic systems. The system can be extended with sensors for smart traffic control.